分块

- lk

前言

- 若无特殊说明数据范围为10^5。

序列分块

- 把序列剖分成$\sqrt{n}$个一块，然后每块维护整块的信息，然后区间询问就会变成$O(\sqrt{n})$个散块和$O(\sqrt{n})$个整块。
- 类似于一个三层的线段树。

询问分块

- 把询问离线下来然后分块，按顺序每次处理一个块内询问，处理完之后把整块的修改作用到序列上(重构？)然后处理下一个块。

根号平衡

- 可以利用均值不等式来调整块大小使得复杂度最优。

莫队

- 1.1讲过了

根号分治

- 大概就是两个不同的算法可以对不同数据选用其中一种来平衡复杂度。

块状链表

块状链表

块状链表

块状链表

块状链表

块状链表

[BZOJ3337]ORZJRY I

输入格式	说明	示例 （例如原序列为 5 2 6 3 1 4）
1 x val	**在第 x 个数后插入一个 val** （0≤x≤序列长度，val>0）	输入：1 2 7 序列：5 2 **7** 6 3 1 4
2 x	**删除第 x 个数** （1≤x≤序列长度）	输入：2 1 序列：**2 6 3 1 4**
3 x y	**翻转第 x 至第 y 个数** （1≤x≤y≤序列长度）	输入：3 3 5 序列：5 2 **1 3 6** 4
4 x y k	**将第 x 至第 y 个数旋转（向右移动）k 次** （1≤x≤y≤序列长度，1≤k≤y-x）	输入：4 1 6 1 序列：**4 5 2 6 3 1**
5 x y val	**将第 x 至第 y 个数加上 val** （1≤x≤y≤序列长度，val>0）	输入：5 3 4 5 序列：5 2 **11 8** 1 4
6 x y val	**将第 x 至第 y 个数都修改为 val** （1≤x≤y≤序列长度，val>0）	输入：6 1 4 7 序列：**7 7 7 7** 1 4
7 x y	**询问第 x 至第 y 个数的和** （1≤x≤y≤序列长度）	输入：7 2 4 输出：**11**
8 x y	**询问第 x 至第 y 个数中最大值与最小值的差** （1≤x≤y≤序列长度）	输入：8 1 3 输出：**4**
9 x y val	**询问第 x 至第 y 个数中与 val 的差的绝对值的最小值** （1≤x≤y≤序列长度，val>0）	输入：9 2 4 5 输出：**1**
10 x y k	**询问第 x 至第 y 个数中第 k 小的数** （1≤x≤y≤序列长度，1≤k≤y-x+1）	输入：10 1 6 4 输出：**4**
11 x y val	**询问第 x 至第 y 个数中比 val 小的数的个数** （1≤x≤y≤序列长度，val>0）	输入：11 2 5 4 输出：**3**

$n, q \leq 10^5, 10s$ (3个点 30s)

[BZOJ3337]ORZJRY I

- 每个块维护一下原数组和排序后的数组，分裂和合并在第二个序列上只需要分离和归并。这样前面一些操作都是打标记。
- 7和8也可以直接遍历，9 11需要在序列中二分，10需要多一个二分答案，复杂度$O(B+\frac{n}{B}\log^2 n)$，根号平衡取$B=\sqrt{n}\log n$后复杂度为$O(q\sqrt{n}\log n)$

[AHOI2013]作业

- 多次查询区间[l,r]中值在[a,b]内的不同数个数。
- $n \leq 10^5, q \leq 10^6$。
- 不要树套树不要树套树不要树套树不要树套树不要树套树不要树套树不要树套树不要树套树不要树套树

[AHOI2013]作业

- 直接莫队+树状数组，时间复杂度$O(n\sqrt{q}\log n)$。
- 注意到修改次数$O(n\sqrt{q})$查询次数$O(q)$，使用$O(1)$修改$O(\sqrt{n})$查询的分块即可做到$O(n\sqrt{q}+q\sqrt{n})$

区间逆序对

区间逆序对

- 给定一个序列，q次询问一个区间的逆序对个数。

区间逆序对

- 给定一个序列，q次询问一个区间的逆序对个数。
- $n, q \leq 10^5$

区间逆序对

- 给定一个序列，q次询问一个区间的逆序对个数。
- $n, q \leq 10^5$
- 要求复杂度不能带log

二次离线莫队

二次离线莫队

- 首先我们知道这个题可以莫队加树状数组$O(n\sqrt{q}\log n)$。

二次离线莫队

- 首先我们知道这个题可以莫队加树状数组$O(n\sqrt{q}\log n)$。
- 发现需要询问$n\sqrt{q}$次 形如“求$[l, r]$内 $< a_x$的数个数”的询问。

二次离线莫队

- 首先我们知道这个题可以莫队加树状数组$O(n\sqrt{q}\log n)$。
- 发现需要询问$n\sqrt{q}$次 形如“求$[l, r]$内 $< a_x$的数个数”的询问。
- 再进行一次差分变成询问$1\ldots x$内 $< y$的数个数，这样可以离线下来对x扫描线，然后变成$O(n)$次修改$O(n\sqrt{q})$次询问区间和，使用$\sqrt{n}$修改$O(1)$询问的分块就可以做到sqrt不带log的复杂度。

二次离线莫队

- 首先我们知道这个题可以莫队加树状数组$O(n\sqrt{q}\log n)$。
- 发现需要询问$n\sqrt{q}$次 形如“求$[l, r]$内 $< a_x$的数个数”的询问。
- 再进行一次差分变成询问$1\ldots x$内 $< y$的数个数，这样可以离线下来对x扫描线，然后变成$O(n)$次修改$O(n\sqrt{q})$次询问区间和，使用$\sqrt{n}$修改$O(1)$询问的分块就可以做到sqrt不带log的复杂度。
- 不过这个做法的问题在于空间复杂度$O(n\sqrt{q})$。

二次离线莫队

- 首先我们知道这个题可以莫队加树状数组$O(n\sqrt{q}\log n)$。
- 发现需要询问$n\sqrt{q}$次 形如“求$[l, r]$内 $< a_x$的数个数”的询问。
- 再进行一次差分变成询问$1\ldots x$内 $< y$的数个数，这样可以离线下来对x扫描线，然后变成$O(n)$次修改$O(n\sqrt{q})$次询问区间和，使用$\sqrt{n}$修改$O(1)$询问的分块就可以做到sqrt不带log的复杂度。
- 不过这个做法的问题在于空间复杂度$O(n\sqrt{q})$。
- 注意到，`while(r<R)++r;` 这个过程的询问全都满足l相等且$r = x - 1$，差分后r的部分可以预处理，l-1的部分可以存一个区间，这样就能节省空间。(`--r`, `++l`, `--l`类似)

[YNOI2015] 盼君勿忘

[YNOI2015] 盼君勿忘

- 给定一个序列，每次给定l, r, p, 查询一个区间$[l, r]$所有子序列分别去重后的和 $\bmod p$。

[YNOI2015] 盼君勿忘

- 给定一个序列，每次给定l, r, p, 查询一个区间$[l, r]$所有子序列分别去重后的和 $\bmod p$。
- $p \leq 10^9$

[YNOI2015] 盼君勿忘

[YNOI2015] 盼君勿忘

- 对每个数x，设出现次数是c，总共长度是l，则x对答案的贡献是$x2^{l-c}(2^c-1)=x(2^l-2^{l-c})$。

[YNOI2015] 盼君勿忘

- 对每个数x，设出现次数是c，总共长度是l，则x对答案的贡献是$x2^{l-c}(2^c-1)=x(2^l-2^{l-c})$。
- 拆成$x2^l$和$x2^{l-c}$分开算，前一部分只要算出区间不同的x的和即可，后者可以发现由于$\sum c=l\le n$，一共只有$O(\sqrt{n})$个不相等的c，只需要算出这些c的x和。

[YNOI2015] 盼君勿忘

- 对每个数x，设出现次数是c，总共长度是l，则x对答案的贡献是$x2^{l-c}(2^c-1)=x(2^l-2^{l-c})$。
- 拆成$x2^l$和$x2^{l-c}$分开算，前一部分只要算出区间不同的x的和即可，后者可以发现由于$\sum c=l\le n$，一共只有$O(\sqrt{n})$个不相等的c，只需要算出这些c的x和。
- 还有一个问题在于快速幂是$O(\log n)$的，可以预处理$2^1\ldots2^{\sqrt{n}}$和$2^{\sqrt{n}},2^{2\sqrt{n}}\ldots2^{(\sqrt{n})^2}$然后$O(1)$询问。

[LUOGU5386] [CNOI2019]数字游戏

[LUOGU5386] [CNOI2019]数字游戏

- 给定一个排列，每次询问一个区间$[l, r]$有多少子区间值全部在区间$[x, y]$内。

数字游戏

数字游戏

- 考虑对着值域区间限制莫队。

数字游戏

- 考虑对着值域区间限制莫队。
- 那么我们对于一个值域区间，把原序列中在这个区间内的记为1其它的记为0，询问的就是区间中每个极长1段长度x, $\binom{x+1}{2}$的和。这就仅和每一段的长度相关。

数字游戏

- 考虑对着值域区间限制莫队。
- 那么我们对于一个值域区间，把原序列中在这个区间内的记为1其它的记为0，询问的就是区间中每个极长1段长度x, $\binom{x+1}{2}$的和。这就仅和每一段的长度相关。
- 考虑再对序列分块，每个块维护一下块内和首尾相连的长度，和其余段的答案。

数字游戏

- 考虑对着值域区间限制莫队。
- 那么我们对于一个值域区间，把原序列中在这个区间内的记为1其它的记为0，询问的就是区间中每个极长1段长度x, $\binom{x+1}{2}$的和。这就仅和每一段的长度相关。
- 考虑再对序列分块，每个块维护一下块内和首尾相连的长度，和其余段的答案。
- 对于每个极长区间的两端记一下另一端的位置就可以做到$O(1)$插入。使用不删除莫队即可。

[POI2015] ODW

- 给定一棵树，每次给x, y, k，求x到y路径上每步走k条边能经过的点点权和，保证$k \mid dist(x, y)$。

[POI2015] ODW

[POI2015] ODW

- 对于$k \le \sqrt{n}$的时候可以离线之后对于每个不同的k $O(n)$计算答案。

[POI2015] ODW

- 对于$k \le \sqrt{n}$的时候可以离线之后对于每个不同的k $O(n)$计算答案。
- 对于$k > \sqrt{n}$的时候最多只有$\sqrt{n}$步，直接树剖/倍增是$\sqrt{n}\log n$，可以根号平衡到$O(q\sqrt{n\log n})$

[POI2015] ODW

- 对于$k \le \sqrt{n}$的时候可以离线之后对于每个不同的k $O(n)$计算答案。
- 对于$k > \sqrt{n}$的时候最多只有$\sqrt{n}$步，直接树剖/倍增是$\sqrt{n}\log n$，可以根号平衡到$O(q\sqrt{n\log n})$
- 优化可以考虑长剖$O(1)$ k级祖先，也可以在树剖的过程中每条重链找出链上的点$O(\sqrt{n}+\log n)$,或者同样是离线下来，用栈记录到根链上的点。

[YNOI2011] 初始化

[YNOI2011] 初始化

1. 对所有 $\bmod y = x$的位置$+z$。

[YNOI2011] 初始化

1. 对所有 $\bmod y = x$的位置$+z$。
2. 求区间和

[YNOI2011] 初始化

[YNOI2011] 初始化

- 根号分治，$y > \sqrt{n}$用$O(1)$修改$O(\sqrt{n})$查询的分块暴力修改。

[YNOI2011] 初始化

- 根号分治，$y > \sqrt{n}$用$O(1)$修改$O(\sqrt{n})$查询的分块暴力修改。
- $y \le \sqrt{n}$的部分，可以记一下$w_{a,b}$表示当前所有$y = a, x = b$的修改的z的和。

[YNOI2011] 初始化

- 根号分治，$y > \sqrt{n}$用$O(1)$修改$O(\sqrt{n})$查询的分块暴力修改。
- $y \le \sqrt{n}$的部分，可以记一下$w_{a,b}$表示当前所有$y = a, x = b$的修改的z的和。
- 询问的时候先拆成前缀和，枚举y，然后每个x的贡献次数一定是形如$a+1, a+1 \ldots a+1, a, a \ldots a$

[YNOI2011] 初始化

- 根号分治，$y > \sqrt{n}$用$O(1)$修改$O(\sqrt{n})$查询的分块暴力修改。
- $y \le \sqrt{n}$的部分，可以记一下$w_{a,b}$表示当前所有$y = a, x = b$的修改的z的和。
- 询问的时候先拆成前缀和，枚举y，然后每个x的贡献次数一定是形如$a+1, a+1 \ldots a+1, a, a \ldots a$
- 也就是拆成全局和 $\times a$加上一个前缀和。都可以在修改的时候预处理前缀和来完成。

[CF896E] WELCOME HOME, CHTHOLLY

[CF896E] WELCOME HOME, CHTHOLLY

- 1>把区间$[l, r]$中 $> x$的$-x$

[CF896E] WELCOME HOME, CHTHOLLY

- 1>把区间$[l, r]$中 $> x$的$-x$
- 2>查询区间$[l, r]$中x的出现次数

[CF896E] WELCOME HOME, CHTHOLLY

- 1>把区间$[l, r]$中 $> x$的$-x$
- 2>查询区间$[l, r]$中x的出现次数
- 值域10^5

[CF896E] WELCOME HOME, CHTHOLLY

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。
- 每个块维护一下每个值的下标列表。

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。
- 每个块维护一下每个值的下标列表。
- 修改的时候对于整块的情况，设当前 $\max = y$，当$y < 2x$的时候可以直接把$x + 1 \ldots y$合并到$1 \ldots y - x$。

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。
- 每个块维护一下每个值的下标列表。
- 修改的时候对于整块的情况，设当前 $\max = y$，当$y < 2x$的时候可以直接把$x + 1 \ldots y$合并到$1 \ldots y - x$。
- 否则我们可以把$1 \ldots x$合并到$x + 1 \ldots 2x$再整体$-x$。

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。
- 每个块维护一下每个值的下标列表。
- 修改的时候对于整块的情况，设当前 $\max = y$，当$y < 2x$的时候可以直接把$x + 1 \ldots y$合并到$1 \ldots y - x$。
- 否则我们可以把$1 \ldots x$合并到$x + 1 \ldots 2x$再整体$-x$。
- 散块可以直接重构。如果合并是$O(1)$的，时间复杂度$O(\sqrt{n})$。

[CF896E] WELCOME HOME, CHTHOLLY

- 分块。
- 每个块维护一下每个值的下标列表。
- 修改的时候对于整块的情况，设当前 $\max = y$，当$y < 2x$的时候可以直接把$x + 1 \ldots y$合并到$1 \ldots y - x$。
- 否则我们可以把$1 \ldots x$合并到$x + 1 \ldots 2x$再整体$-x$。
- 散块可以直接重构。如果合并是$O(1)$的，时间复杂度$O(\sqrt{n})$。
- 显然合并可以直接用并查集合并，就可以了。

[集训队互测2018]完美的队列

[集训队互测2018]完美的队列

- 有n个队列，第i个有一个大小限制a_i。q次操作，每次对一个区间l, r进行q[i].push(x)，并对大小超过限制的队列进行q[i].pop()，每次操作后求所有队列中一共有多少个不同的权值。

[集训队互测2018]完美的队列

[集训队互测2018]完美的队列

- 对每个区间push操作，考虑求出什么时候会全pop掉。然后对于相同的x把区间取个并就行了。

[集训队互测2018]完美的队列

- 对每个区间push操作，考虑求出什么时候会全pop掉。然后对于相同的x把区间取个并就行了。
- 对序列分块，拆成整块和散块的询问。

[集训队互测2018]完美的队列

- 对每个区间push操作，考虑求出什么时候会全pop掉。然后对于相同的x把区间取个并就行了。
- 对序列分块，拆成整块和散块的询问。
- 先考虑整块，可以双指针，记一下w_i为第i个队列还差w_i个push就能把x弹掉，则全弹掉的条件是 $\max\ w_i \leq 0$。

[集训队互测2018]完美的队列

- 对每个区间push操作，考虑求出什么时候会全pop掉。然后对于相同的x把区间取个并就行了。
- 对序列分块，拆成整块和散块的询问。
- 先考虑整块，可以双指针，记一下w_i为第i个队列还差w_i个push就能把x弹掉，则全弹掉的条件是 $\max\ w_i \le 0$。
- 然后每次操作就是区间-1或全局-1，可以发现非全局的-1在所有块中最多出现$O(q)$次，而全局-1最多出现$O(q\sqrt{n})$次，所以对全局打标记，非全局暴力即可。

[集训队互测2018]完美的队列

[集训队互测2018]完美的队列

- 散块也是类似的双指针，但显然不能对所有操作扫一遍。

[集训队互测2018]完美的队列

- 散块也是类似的双指针，但显然不能对所有操作扫一遍。
- 注意到散块的单点总个数也是$O(q\sqrt{n})$，只需要把相邻的相同的整块-1合并即可(快速定位某一个很简单)

[集训队互测2018]完美的队列

- 散块也是类似的双指针，但显然不能对所有操作扫一遍。
- 注意到散块的单点总个数也是$O(q\sqrt{n})$，只需要把相邻的相同的整块-1合并即可(快速定位某一个很简单)
- 时间复杂度$O(q\sqrt{n})$

[CF1545F] AQUAMOON AND POTATOES

[CF1545F] AQUAMOON AND POTATOES

- 给定三个长度为n的数列a, b, c满足$1 \le a_i, b_i, c_i \le n$。

[CF1545F] AQUAMOON AND POTATOES

- 给定三个长度为n的数列a, b, c满足$1 \le a_i, b_i, c_i \le n$。
- q次操作：修改某个a_i或者 给定r询问$\sum_{1 \le i < j < k \le r} [b_{a_i} = a_j = c_{a_k}]$。

[CF1545F] AQUAMOON AND POTATOES

- 给定三个长度为n的数列a, b, c满足$1 \le a_i, b_i, c_i \le n$。
- q次操作：修改某个a_i或者 给定r询问$\sum_{1 \le i < j < k \le r}[b_{a_i} = a_j = c_{a_k}]$。
- $n \le 2 \times 10^5, q \le 5 \times 10^4, 7s$。

[CF1545F] AQUAMOON AND POTATOES

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$，询问$\sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$，询问\sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用$1, 2, 3$位置分别表示作为$\mathrm{i}, \mathrm{j}, \mathrm{k}$。

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$， 询 问 \sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用1, 2, 3位置分别表示作为i, j, k 。
- 先考虑一个线性dp：for(i=1;i<=r;++i) {ans+=f2[c[i]];f2[b[i]]+=f1[b[i]];++f1[a[i]];}

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$，询问$\sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用1, 2, 3位置分别表示作为i, j, k。
- 先考虑一个线性dp：for(i=1;i<=r;++i) {ans+=f2[c[i]];f2[b[i]]+=f1[b[i]];++f1[a[i]];}
- 考虑分块，维护每个块前缀的ans, f1, f2。

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$，询问$\sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用1, 2, 3位置分别表示作为i, j, k。
- 先考虑一个线性dp：for(i=1;i<=r;++i) {ans+=f2[c[i]];f2[b[i]]+=f1[b[i]];++f1[a[i]];}
- 考虑分块，维护每个块前缀的ans, f1, f2。
- 那么修改的时候，就需要算一下包含i的方案数。

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$， 询 问 \sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用1, 2, 3位置分别表示作为i, j, k 。
- 先考虑一个线性dp：for(i=1;i<=r;++i) {ans+=f2[c[i]];f2[b[i]]+=f1[b[i]];++f1[a[i]];}
- 考虑分块，维护每个块前缀的ans, f1, f2。
- 那么修改的时候，就需要算一下包含i的方案数。
- 先考虑当前块内。先加前面的元素，记一个dp[l][r]表示当前确定了l..r位置，然后i填在r号位。到块首之后加上前面前缀的f1,f2。

[CF1545F] AQUAMOON AND POTATOES

- 为了方便表述，下面考虑的题目是，修改$a_{i,1}, a_{i,2}, a_{i,3}$， 询 问 \sum [i\lt j\lt k][a_{i,1}=a_{j,2}=a_{k,3}]$。
- 下面用1, 2, 3位置分别表示作为i, j, k 。
- 先考虑一个线性dp：for(i=1;i<=r;++i) {ans+=f2[c[i]];f2[b[i]]+=f1[b[i]];++f1[a[i]];}
- 考虑分块，维护每个块前缀的ans, f1, f2。
- 那么修改的时候，就需要算一下包含i的方案数。
- 先考虑当前块内。先加前面的元素，记一个dp[l][r]表示当前确定了l..r位置，然后i填在r号位。到块首之后加上前面前缀的f1,f2。
- 加后面的元素的时候类似，就是dp[l][r]表示确定了1..r位置，i填在l号位。

[CF1545F] AQUAMOON AND POTATOES

[CF1545F] AQUAMOON AND POTATOES

- 算往后面的块的贡献的时候，发现我们还需要每个块多维护一个`s[l..r][c]`表示填了l..r号位了，值是c。

[CF1545F] AQUAMOON AND POTATOES

- 算往后面的块的贡献的时候，发现我们还需要每个块多维护一个`s[l..r][c]`表示填了l..r号位了，值是c。
- 但实际上l..r只有2,2 3,3 2,3 是有用的。

[CF1545F] AQUAMOON AND POTATOES

- 算往后面的块的贡献的时候，发现我们还需要每个块多维护一个`s[l..r][c]`表示填了l..r号位了，值是c。
- 但实际上l..r只有2,2 3,3 2,3 是有用的。
- 假设我们有了这个s，就可以类似的用dp维护f1,f2,ans的增量。

[CF1545F] AQUAMOON AND POTATOES

- 算往后面的块的贡献的时候，发现我们还需要每个块多维护一个`s[l..r][c]`表示填了l..r号位了，值是c。
- 但实际上l..r只有2,2 3,3 2,3 是有用的。
- 假设我们有了这个s，就可以类似的用dp维护f1,f2,ans的增量。
- 然后s怎么维护呢，注意只有这3种情况，讨论一下就行了。

[CF1545F] AQUAMOON AND POTATOES

- 算往后面的块的贡献的时候，发现我们还需要每个块多维护一个`s[l..r][c]`表示填了l..r号位了，值是c。
- 但实际上l..r只有2,2 3,3 2,3 是有用的。
- 假设我们有了这个s，就可以类似的用dp维护f1,f2,ans的增量。
- 然后s怎么维护呢，注意只有这3种情况，讨论一下就行了。
- 至于如果要卡空间的话，可以把这个东西离线下来一块一块做，但是会稍微麻烦一点。

[CF1588F] JUMPING THROUGH THE ARRAY

[CF1588F] JUMPING THROUGH THE ARRAY

- 有一个排列p_i和一个数组a_i。

[CF1588F] JUMPING THROUGH THE ARRAY

- 有一个排列p_i和一个数组a_i。

1. 求a区间和

[CF1588F] JUMPING THROUGH THE ARRAY

- 有一个排列p_i和一个数组a_i。

1. 求a区间和
2. 给$x, p_x, p_{p_x} \ldots$这个环上的所有位置y的$a_y + z$。

[CF1588F] JUMPING THROUGH THE ARRAY

- 有一个排列p_i和一个数组a_i。

1. 求a区间和
2. 给$x, p_x, p_{p_x} \ldots$这个环上的所有位置y的$a_y + z$。
3. 交换p_x, p_y

[CF1588F] JUMPING THROUGH THE ARRAY

- 有一个排列p_i和一个数组a_i。

1. 求a区间和
2. 给$x, p_x, p_{p_x} \ldots$这个环上的所有位置y的$a_y + z$。
3. 交换p_x, p_y

- $n, q \leq 2 \times 10^5, 8s$

[CF1588F] JUMPING THROUGH THE ARRAY

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法1：先序列分块。然后对每个环用块状链表维护。散点求值可以直接定位到块。

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法1：先序列分块。然后对每个环用块状链表维护。散点求值可以直接定位到块。
- 整块的部分，对于环的块状链表只有一段的，这一段有可能 $< \frac{B}{2}$，我们只需要维护这些段加起来，原序列每一块的和，然后询问的时候直接区间求和就行。

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法1：先序列分块。然后对每个环用块状链表维护。散点求值可以直接定位到块。
- 整块的部分，对于环的块状链表只有一段的，这一段有可能 $< \frac{B}{2}$，我们只需要维护这些段加起来，原序列每一块的和，然后询问的时候直接区间求和就行。
- 对于块状链表有超过一段的，每一段都在$\frac{B}{2}$和$2B$之间，我们对每一段维护这一段在每一块内的个数的前缀和，和的前缀和，然后询问的时候枚举这些段O(1)求区间和就行。

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法1：先序列分块。然后对每个环用块状链表维护。散点求值可以直接定位到块。
- 整块的部分，对于环的块状链表只有一段的，这一段有可能 $< \frac{B}{2}$，我们只需要维护这些段加起来，原序列每一块的和，然后询问的时候直接区间求和就行。
- 对于块状链表有超过一段的，每一段都在$\frac{B}{2}$和$2B$之间，我们对每一段维护这一段在每一块内的个数的前缀和，和的前缀和，然后询问的时候枚举这些段O(1)求区间和就行。
- 时间复杂度$O(n\sqrt{n})$

[CF1588F] JUMPING THROUGH THE ARRAY

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法2:先询问分块。然后考虑处理一块内的答案，则只会有sqrt个点的出边变化，不变的就可以直接缩点，会缩成sqrt个点。同时询问也可以拆成sqrt段，每次询问是一个段的区间和。

[CF1588F] JUMPING THROUGH THE ARRAY

- 算法2:先询问分块。然后考虑处理一块内的答案，则只会有sqrt个点的出边变化，不变的就可以直接缩点，会缩成sqrt个点。同时询问也可以拆成sqrt段，每次询问是一个段的区间和。
- 为了询问区间和，我们就需要对缩成的sqrt个点每个点记录一下在每一段内的点数和和，询问的时候枚举每个点求贡献和，为了减小复杂度，在前面一部分记的时候改成记前缀和，这样就能根号了。

THANKS!